



The Islamic University of
Gaza Faculty of Engineering
Department of Computer Engineering



ECOM 1101 - Digital Design

Lab #4

Combinational Logic

Eng. Hashem M. Hejazy

Eng. Rania R. Alsayed

Spring 2025/2026

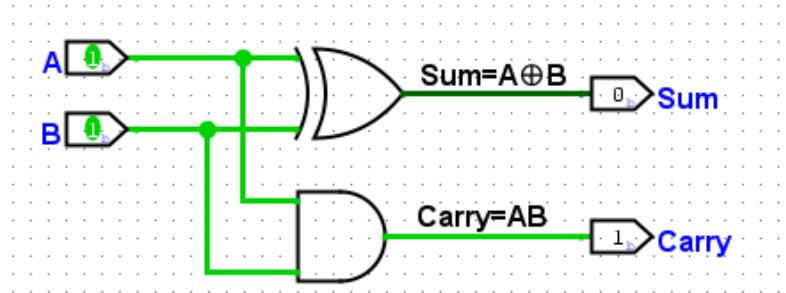
Introduction

Digital computers perform a variety of information-processing tasks. Among the functions encountered are the various arithmetic operations. The most basic arithmetic operation is the addition of two binary digits. This simple addition consists of four possible elementary operations: $0 + 0 = 0$, $0 + 1 = 1$, $1 + 0 = 1$, and $1 + 1 = 10$. The first three operations produce a sum of one digit, but when both augend and addend bits are equal to 1, the binary sum consists of two digits. The higher significant bit of this result is called a *carry*. When the augend and addend numbers contain more significant digits, the carry obtained from the addition of two bits is added to the next higher order pair of significant bits. A combinational circuit that performs the addition of two bits is called a **half adder**. One that performs the addition of three bits (two significant bits and a previous carry) is a **full adder**.

Half Adder

Half adder is a combinational arithmetic circuit that adds two numbers and produces a sum bit (S) and carry bit (C) as the output.

Inputs		Output	
A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1



From this it is clear that a half adder circuit can be easily constructed using one XOR gate and one AND gate. Half adder is the simplest of all adder circuits, but it has a major disadvantage, the half adder can add only two input bits (A and B) and has nothing to do with the carry if there is any in the input, so if the input to a half adder have a carry, then it will be neglected and adds only the A and B bits.

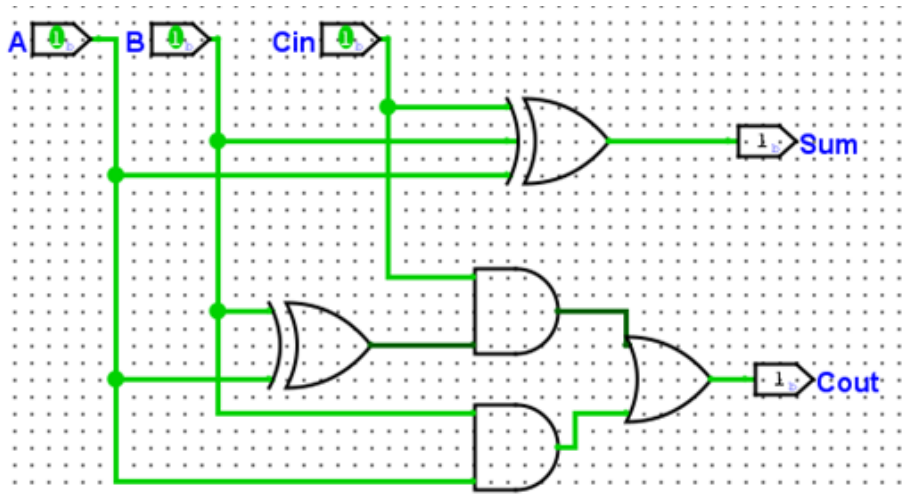
Full Adder

Full adder is a logic circuit that adds two input operand bits plus a Carry in bit and outputs a Carry out bit and a sum bit. The Sum out (Sout) of a full adder is the XOR of input operand bits A, B and the Carry in (Cin) bit. Truth table and logic diagram of a 1-bit Full adder is shown below.

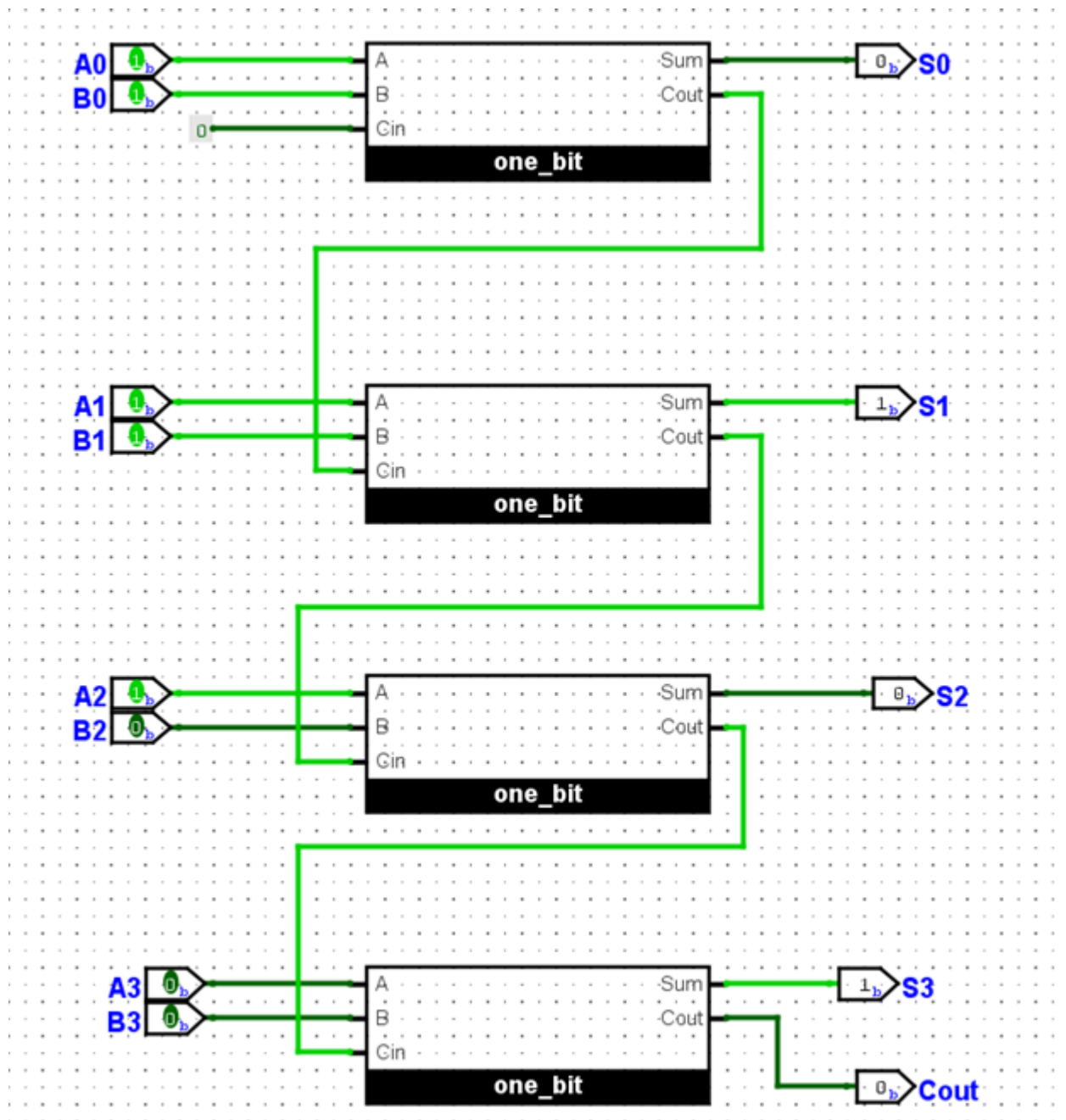
Inputs			Outputs	
A	B	Cin	S	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$$S = A'B'Cin + A'BCin' + AB'Cin' + ABCin = A \oplus B \oplus Cin$$

$$Cout = ABCin' + AB'Cin + A'BCin + ABCin = (A \oplus B)Cin + AB$$



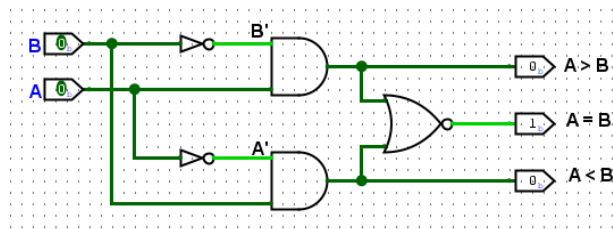
4-bit Full Adder



Comparators:

A single bit comparator is just a design that compares two bits if the two bits and this design have three outputs ($A > B$, $A < B$, and $A = B$), so according to the values of the bits, the output will be zero or one.

A	B	$A > B$	$A < B$	$A = B$
0	0	0	0	1
0	1	0	1	0
1	0	1	0	0
1	1	0	0	1



Decoders

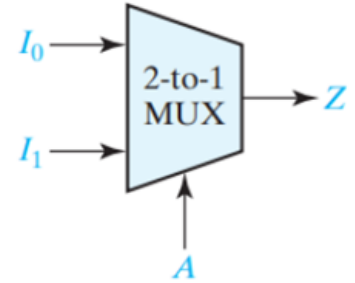
A decoder is a logic circuit that will detect the presence of a specific binary number or word. The input to the decoder is a parallel binary number and the output is a binary signal that indicates the presence or absence of that specific number. It is a combinational circuit that converts binary information from n input lines to a maximum of 2^n unique output lines

2-to-4 decoder:

Inputs		Outputs			
B	A	F0	F1	F2	F3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

Multiplexer

A multiplexer (or data selector, abbreviated as MUX) has a group of data inputs and a group of control inputs. The control inputs are used to select one of the data inputs and connect it to the output terminal as shown in figure.

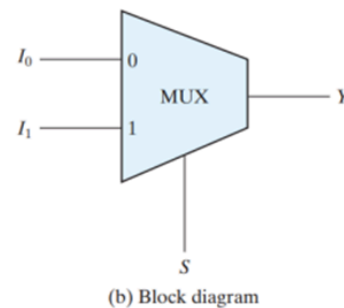
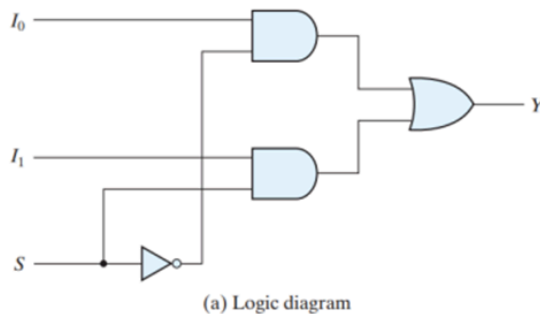


Multiplexer Use:

The main use is as a transmitter for data between input and output depending on the selection.

Multiplexer Types:

The type of mux depend on the number of the selection line that it had as follow 2^n -to-1 where n is the number of selection line.



2-to-1 Mux

From the previous figure: $Y = I_0 S' + I_1 S$

and the truth table will be as follow:

S	I ₁	I ₀	Y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

OR:

S	Y
0	I ₀
1	I ₁

Implementing functions with multiplexers:

Solution Step

1. Select the type of Mux [2^{n-1} -to-1].
2. Select (n-1) as selection line.
3. The other input connects as input

Example: Implement the following Boolean function with a multiplexer

$$F(A, B, C, D) = \pi(0, 1, 5, 7)$$

Solution:

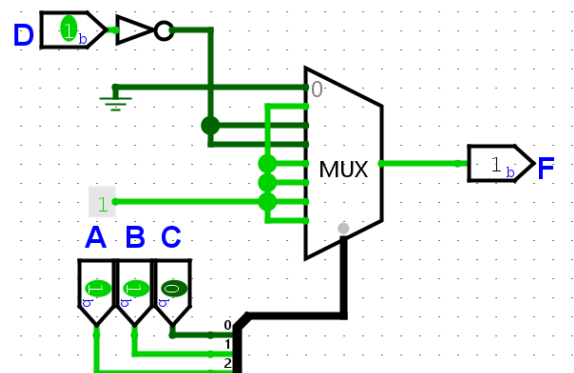
1. $n=4$ so we chose mux ($2^{(4-1)}$ -to-1) that's mean we need mux 8 – to-1.
2. Chose $(n-1)$ as select line, so we chose 3 select line which are(A,B and C).
3. D input line will connect as input.

A	B	C	D	F
0	0	0	0	0
0	0	0	1	0
0	0	1	0	1
0	0	1	1	1
0	1	0	0	1
0	1	0	1	0
0	1	1	0	1
0	1	1	1	0
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

From the above truth table, we can found this

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	D'
0	1	1	D'
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

In the simulation, its look like this:



Ex: Find the function of this table and draw it.

A	B	S	Y
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Solution:

-10Collapse Duplicated RowsShow All Rows

8 of 8 rows shown

A	B	S	Y
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Output:

Format:

Style:

Notation:

		B, S			
		00	01	11	10
A	0	0	1	0	1
	1	0	1	1	1

No group selected.

$$\overline{B} \cdot S + B \cdot \overline{S} + A \cdot S$$

$$Y = B'S + BS' + AS$$

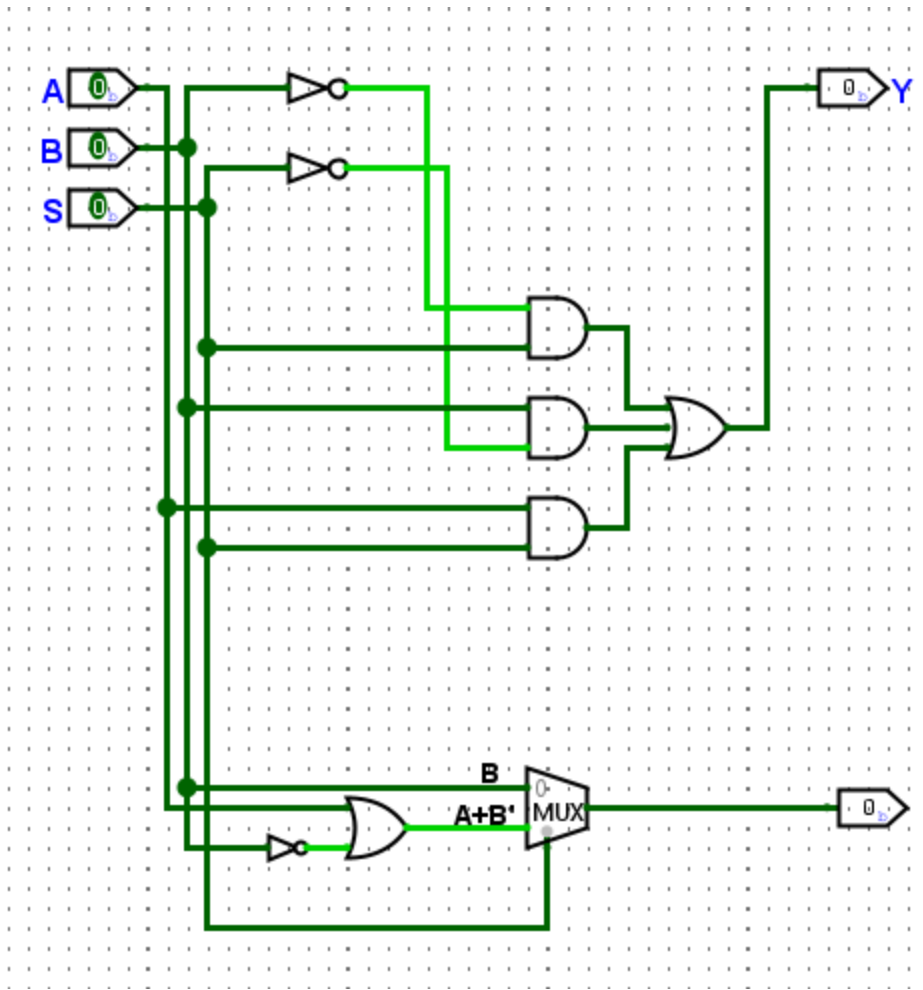
$$Y = S(A+B') + BS'$$

Back to the general form of MUX

$$Y = I_0S' + I_1S$$

$$I_0 = B', I_1 = (A+B')$$

So, you can use MUX to draw the final function.



Homework

Q1: Construct a 4-to-16- line decoder with five 2-to-4- line decoders with enable

Q2: Design a combinational circuit that compares two 4-bit numbers to check if they are equal. The circuit output is equal to 1 if the two numbers are equal and 0 otherwise.

Q3: An 8*1 multiplexer has inputs A, B, and C connected to the selection inputs S2,S1, and S0, respectively. The data inputs

$$I_1=I_2=I_7=0; \quad I_3=I_5=1; \quad I_0=I_4=D; \quad \text{and } I_6=D'$$